

Universidad Rafael Landívar
Facultad de Ingeniería
Licenciatura en Ingeniería en Informática y Sistemas
Lenguajes Formales y Autómatas
Docente: Ing. Cesar Rojas

Proyecto No.1

“Motor de Parsing, Validaciones Estructurales y Simulación de Autómatas Finitos Deterministas (AFD)”

Estudiantes: Alessandro López, Sebastián
Alexander Villeda Reyna
Carné: 1187525, 1032625

Guatemala 30 de agosto de 2026

1. Fundamento teórico y objetivo

Un Autómata Finito Determinista (AFD) se define formalmente mediante la quintupla $M = (Q, \Sigma, \delta, q_0, F)$, donde Q es el conjunto finito de estados, Σ es el alfabeto de entrada, $\delta: Q \times \Sigma \rightarrow Q$ es la función de transición, $q_0 \in Q$ es el estado inicial y $F \subseteq Q$ es el conjunto de estados de aceptación. Una cadena w es aceptada si, partiendo de q_0 y aplicando δ símbolo a símbolo, se alcanza un estado perteneciente a F . El lenguaje $L(M)$ reconocido por el autómata es el conjunto de todas las cadenas aceptadas, y es vacío si y solo si ningún estado de F es alcanzable desde q_0 .

El objetivo del proyecto es implementar en Python, mediante consola, un sistema capaz de construir, validar y simular un AFD a partir de su definición formal, ya sea por ingreso interactivo o por lectura de un archivo de texto plano analizado con expresiones regulares.

1.1 Formato del archivo de entrada

El sistema reconoce archivos .txt con la siguiente estructura obligatoria:

```
NOMBRE: MiAutomata
ESTADOS: q0,q1,q2
ALFABETO: a,b
INICIAL: q0
FINALES: q2
TRANSICIONES:
q0,a,q1
q0,b,q0
q1,a,q1
q1,b,q2
q2,a,q2
q2,b,q2
```

Cada sección se identifica con una etiqueta seguida de dos puntos. La sección TRANSICIONES: se extiende hasta el final del archivo y cada línea posterior debe cumplir el patrón 'origen,símbolo,destino'.

1.2 Expresiones regulares utilizadas (módulo re)

```
PATRON_NOMBRE = ^\s*NOMBRE\s*:\s*(.+)\s*$
PATRON_ESTADOS = ^\s*ESTADOS\s*:\s*(.+)\s*$
PATRON_ALFABETO = ^\s*ALFABETO\s*:\s*(.+)\s*$
PATRON_INICIAL = ^\s*INICIAL\s*:\s*(.+)\s*$
PATRON_FINALES = ^\s*FINALES\s*:\s*(.+)\s*$
PATRON_BLOQUE_TRANSICIONES = ^\s*TRANSICIONES\s*:\s*(.*) [MULTILINE|DOTALL]
PATRON_LINEA_TRANSICION = ^\s*([^\s,]+)\s*,\s*([^\s,]+)\s*,\s*([^\s,]+)\s*$
PATRON_ID (nombres de estado) = ^[A-Za-z_][A-Za-z0-9_]*$
PATRON_SIMBOLO_ALFABETO (símbolos  $\Sigma$ ) = ^[\s,]+\s$
```

Se utilizan dos patrones distintos porque estados y símbolos siguen convenciones diferentes.

PATRON_ID valida los nombres de estado: deben comenzar con una letra o guion bajo, seguidos de letras, dígitos o guiones bajos (por ejemplo, 'q0' es válido pero '2q' no lo es, ya que un estado se nombra como un identificador). PATRON_SIMBOLO_ALFABETO es deliberadamente más permisivo: acepta cualquier token que no contenga coma ni espacio, lo que permite alfabetos numéricos como $\Sigma = \{0,1\}$ (dígitos puros, sin letra inicial). En ambos casos, los tokens que no cumplen el patrón se reportan como inválidos y se descartan sin detener la carga del archivo.

Restricción verificada experimentalmente: aunque PATRON_SIMBOLO_ALFABETO admite símbolos de más de un carácter (p. ej. $\Sigma = \{10,20,30\}$), el motor de simulación (simulador.py) procesa la cadena de

entrada carácter por carácter con `for simbolo in cadena`. Esto significa que solo los símbolos de UN carácter funcionan correctamente durante la evaluación: un símbolo multi-carácter se carga sin error, pero cualquier cadena evaluada sobre él será rechazada porque ningún carácter individual de la cadena coincide con el símbolo completo registrado en Σ . Esta es la razón por la que, en la práctica, el alfabeto Σ debe estar compuesto por símbolos de un solo carácter (letras, dígitos o signos individuales) — consistente con la definición clásica de AFD usada en el curso.

2. Acciones del programa

El sistema presenta un menú de consola con las siguientes diez acciones:

- 1. Crear un AFD manualmente:** Solicita interactivamente cada elemento de la quintupla $(Q, \Sigma, q_0, F, \delta)$, validando duplicados y referencias inválidas en el momento del ingreso.
- 2. Cargar un AFD desde archivo .txt:** Lee y analiza el archivo mediante expresiones regulares, reportando cada error de sintaxis con su número de línea sin detener la carga.
- 3. Mostrar la definición formal:** Imprime la quintupla $M = (Q, \Sigma, \delta, q_0, F)$ del autómata activo.
- 4. Mostrar la tabla de transición:** Construye una matriz estados $\times$ símbolos con δ , marcando el estado inicial y los finales.
- 5. Validar la estructura:** Ejecuta las tres validaciones formales (estados, consistencia de transiciones, determinismo/completitud), reporta alcanzabilidad y ofrece completar el AFD con un estado trampa si está incompleto.
- 6. Evaluar una cadena:** Solicita una cadena y muestra su traza de ejecución paso a paso, junto con el veredicto (ACEPTADA/RECHAZADA) y su motivo.
- 7. Evaluar un archivo de cadenas:** Procesa un lote de cadenas leídas de un archivo, generando un reporte consolidado con el total de aceptadas y rechazadas.
- 8. Consultar historial:** Muestra todas las evaluaciones realizadas durante la sesión, en orden cronológico.
- 9. Cargar o crear otro autómata:** Descarta el autómata activo y permite crear uno nuevo (manual o por archivo), reiniciando el historial.
- 10. Salir:** Finaliza la ejecución del programa de forma controlada.

```
=====
SIMULADOR DE AUTÓMATA FINITO DETERMINISTA (AFD)
=====
1. Crear un AFD manualmente
2. Cargar un AFD desde un archivo .txt
3. Mostrar la definición formal del AFD (Quintupla)
4. Mostrar la tabla de transición
5. Validar la estructura del autómata
6. Evaluar una cadena (con traza paso a paso)
7. Evaluar un archivo de cadenas en lote
8. Consultar el historial de evaluaciones
9. Cargar o crear otro autómata
10. Salir
=====
Seleccione una opción (1-10): █
```

3. Datos de entrada

Información solicitada al usuario en cada punto del programa y su tipo de dato:

Dato solicitado	Tipo	¿Dónde se solicita?
Opción de menú	Entero (1-10), validado por rango	Menú principal
Nombre del autómata	Cadena de texto libre	Creación manual / NOMBRE: en archivo
Conjunto de estados Q	Lista de identificadores separados por coma	Creación manual / ESTADOS: en archivo
Alfabeto Σ	Lista de identificadores separados por coma	Creación manual / ALFABETO: en archivo
Estado inicial q_0	Identificador (debe pertenecer a Q)	Creación manual / INICIAL: en archivo
Estados finales F	Lista de identificadores (subconjunto de Q)	Creación manual / FINALES: en archivo
Transiciones δ	Terna origen,símbolo,destino	Creación manual / TRANSICIONES: en archivo
Ruta de archivo .txt	Cadena de texto (ruta del sistema de archivos)	Opciones 2 y 7
Cadena a evaluar	Cadena de símbolos (puede ser vacía, ϵ)	Opción 6
Confirmación estado trampa	Cadena 's'/'n'	Opción 5, si el AFD está incompleto

En el programa esto se ve como:

```
Programa (python\python312\python.exe C:/Users/veget/OneDrive/Escritorio/Proyecto_Lenguajes_Sebastian_y_Alessandro/Proyecto_
in.py
Seleccione una opcion (1-10): 2

Ruta del archivo .txt a cargar: C:\Users\veget\Downloads\Prueba1.txt

[OK] AFD 'AFD_Binario' cargado correctamente desde 'C:\Users\veget\Downloads\Prueba1.txt'.

=====
SIMULADOR DE AUTÓMATA FINITO DETERMINISTA (AFD)
=====
1. Crear un AFD manualmente
2. Cargar un AFD desde un archivo .txt
3. Mostrar la definición formal del AFD (Quíntupla)
4. Mostrar la tabla de transición
5. Validar la estructura del autómata
6. Evaluar una cadena (con traza paso a paso)
7. Evaluar un archivo de cadenas en lote
8. Consultar el historial de evaluaciones
9. Cargar o crear otro autómata
10. Salir
=====
Seleccione una opción (1-10): 3

Autómata: AFD_Binario
Q (Estados)      = {q0, q1}
Sigma (Alfabeto) = {0, 1}
q0 (Estado inicial) = q0
F (Estados finales)= {q1}
delta (Transiciones):
  delta(q0, 0) = q0
  delta(q0, 1) = q1
  delta(q1, 0) = q0
  delta(q1, 1) = q1
```

4. Variables principales del sistema

Variable	Tipo (Python)	Descripción	Módulo
estados (Q)	Set[str]	Conjunto de estados del autómata	afd.py
alfabeto (Σ)	Set[str]	Conjunto de símbolos válidos de entrada	afd.py
transiciones (δ)	Dict[Tuple[str,str], str]	Función de transición (estado,símbolo) $\rightarrow$ estado	afd.py
estado_inicial (q0)	str	Estado de partida de la computación	afd.py
estados_finales (F)	Set[str]	Estados de aceptación	afd.py
transiciones_conflictivas	Dict[Tuple[str,str], Set[str]]	Registro de pares con más de un destino (indicio de AFND)	cargador.py / afd.py
afd_actual	Optional[AFD]	Autómata actualmente cargado en memoria	main.py
estado_actual	str	Estado en curso durante la simulación de una cadena	simulador.py
traza	List[str]	Lista de pasos [estado] --(símbolo)--> [estado] de la computación	simulador.py
historial	List[ResultadoEvaluacion]	Registro acumulado de todas las evaluaciones de la sesión	simulador.py
advertencias	List[str]	Mensajes de error/advertencia acumulados durante el parseo	cargador.py

Ejemplos:

```
=====
SIMULADOR DE AUTÓMATA FINITO DETERMINISTA (AFD)
=====
1. Crear un AFD manualmente
2. Cargar un AFD desde un archivo .txt
3. Mostrar la definición formal del AFD (Quíntupla)
4. Mostrar la tabla de transición
5. Validar la estructura del autómata
6. Evaluar una cadena (con traza paso a paso)
7. Evaluar un archivo de cadenas en lote
8. Consultar el historial de evaluaciones
9. Cargar o crear otro autómata
10. Salir
=====
Seleccione una opción (1-10): 6

Ingrese la cadena a evaluar (ENTER para cadena vacía ε): 0001010101111

--- TRAZA DE EJECUCIÓN: '0001010101111' ---
[q0] --(0)--> [q0]
[q0] --(0)--> [q0]
[q0] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(1)--> [q1]
Veredicto final: ACEPTADA
Motivo: El estado final de la computación 'q1' pertenece a F.
```

5. Condiciones, restricciones y cálculos

5.1 Validación de la quíntupla

- $q_0 \in Q$ — el estado inicial debe pertenecer al conjunto de estados.
- $F \subseteq Q$ — todo estado final declarado debe pertenecer a Q .
- Para toda transición (origen, símbolo, destino): origen $\in Q$, símbolo $\in \Sigma$, destino $\in Q$.

5.1.1 Validación de identificadores (estados vs. símbolos)

- Nombres de estado: deben cumplir PATRON_ID (letra o '_' inicial, seguido de letras/dígitos/'_'). Un estado compuesto solo por dígitos, p. ej. '0', '1' o '2', se rechaza como identificador inválido y se descarta (verificado con un archivo de prueba cuyos ESTADOS eran puramente numéricos: los tres fueron rechazados, Q quedó vacío y el sistema lo reportó sin colapsar).
- Símbolos del alfabeto: se validan con PATRON_SIMBOLO_ALFABETO, más permisivo que PATRON_ID, ya que sí admite tokens que comienzan con dígito. Esto permite alfabetos como $\Sigma = \{0,1\}$ (verificado: un AFD con $\Sigma = \{0,1\}$ carga y simula correctamente cadenas binarias como '0101', '111', '000').
- Limitación verificada: Σ debe estar compuesto por símbolos de EXACTAMENTE un carácter. Un símbolo de varios caracteres (p. ej. $\Sigma = \{10,20,30\}$) se acepta durante la carga del archivo, pero el motor de simulación evalúa la cadena de entrada carácter por carácter, por lo que ninguna cadena podrá coincidir con un símbolo multi-carácter y siempre será rechazada. Esta restricción es consistente con la definición clásica de AFD trabajada en el curso, donde Σ es un alfabeto de símbolos individuales.

5.2 Determinismo y completitud

- Determinismo: para cada par $(q, a) \in Q \times \Sigma$ debe existir a lo sumo una transición. Si existen dos o más destinos distintos para el mismo par, se reporta como transición múltiple (indicio de AFND).
- Completitud: el AFD es completo si, para todo $(q, a) \in Q \times \Sigma$, $\delta(q, a)$ está definida. Los pares sin transición se reportan como 'transición faltante'.
- Transiciones- ϵ : cualquier símbolo de transición perteneciente al conjunto $\{\epsilon, \text{'epsilon'}, \text{'eps'}, \text{'\epsilon'}, \text{'landa'}, \text{'\lambda'}, \text{'lambda'}\}$ se rechaza explícitamente, ya que una transición que no consume símbolo corresponde a un AFND- ϵ , no a un AFD.
- Estado trampa (completitud asistida): dado el conjunto de pares faltantes $P = \{(q,a) : \delta(q,a) \text{ no definida}\}$, se agrega un estado $q_{\text{trampa}} \notin Q$ tal que $\delta(q,a) = q_{\text{trampa}} \forall (q,a) \in P$, y $\delta(q_{\text{trampa}}, a) = q_{\text{trampa}} \forall a \in \Sigma$. Como $q_{\text{trampa}} \notin F$, el lenguaje reconocido no cambia.

5.3 Análisis de alcanzabilidad (BFS)

- Alcanzables(q_0) se calcula mediante recorrido en anchura sobre el grafo implícito de δ , partiendo de q_0 .
- Inaccesibles = $Q - \text{Alcanzables}(q_0)$.
- Finales_alcanzables = $F \cap \text{Alcanzables}(q_0)$.
- $L(M) = \emptyset \iff \text{Finales_alcanzables} = \emptyset$ (ningún estado final es alcanzable desde q_0).

```
Seleccione una opción (1-10): 5
```

```
=== REPORTE DE VALIDACIÓN: AFD_PruebaError ===
```

```
[QUÍNTUPLA] Consistente:  $q_0 \in Q$ ,  $F \subseteq Q$ , transiciones válidas.
```

```
[DETERMINISMO] El autómata es un AFD válido y completo.
```

```
[ALCANZABILIDAD] Estados alcanzables desde  $q_0$ : ['q0', 'q1']
```

```
[ALCANZABILIDAD] Estados inaccesibles ( $Q - \text{alcanzables}$ ): []
```

```
[ALCANZABILIDAD] Estados finales alcanzables: ['q1']
```

```
[LENGUAJE]  $L(M)$  NO es vacío: existe al menos un estado final alcanzable.
```

5.4 Evaluación de una cadena

- Una cadena w se rechaza inmediatamente si contiene algún símbolo que no pertenece a Σ .
- Para cada símbolo a de w , se calcula $\text{estado_actual} = \delta(\text{estado_actual}, a)$; si δ no está definida para ese par, la cadena se rechaza (autómata incompleto en ese punto).
- Al finalizar w , la cadena es ACEPTADA si y solo si $\text{estado_actual} \in F$; en caso contrario, es RECHAZADA.
- La cadena vacía (ϵ) se evalúa sin ejecutar transición alguna: es aceptada si y solo si $q_0 \in F$.

```
=====  
SIMULADOR DE AUTÓMATA FINITO DETERMINISTA (AFD)  
=====
```

1. Crear un AFD manualmente
2. Cargar un AFD desde un archivo .txt
3. Mostrar la definición formal del AFD (Quíntupla)
4. Mostrar la tabla de transición
5. Validar la estructura del autómata
6. Evaluar una cadena (con traza paso a paso)
7. Evaluar un archivo de cadenas en lote
8. Consultar el historial de evaluaciones
9. Cargar o crear otro autómata
10. Salir

```
=====
```

Seleccione una opción (1-10): 2

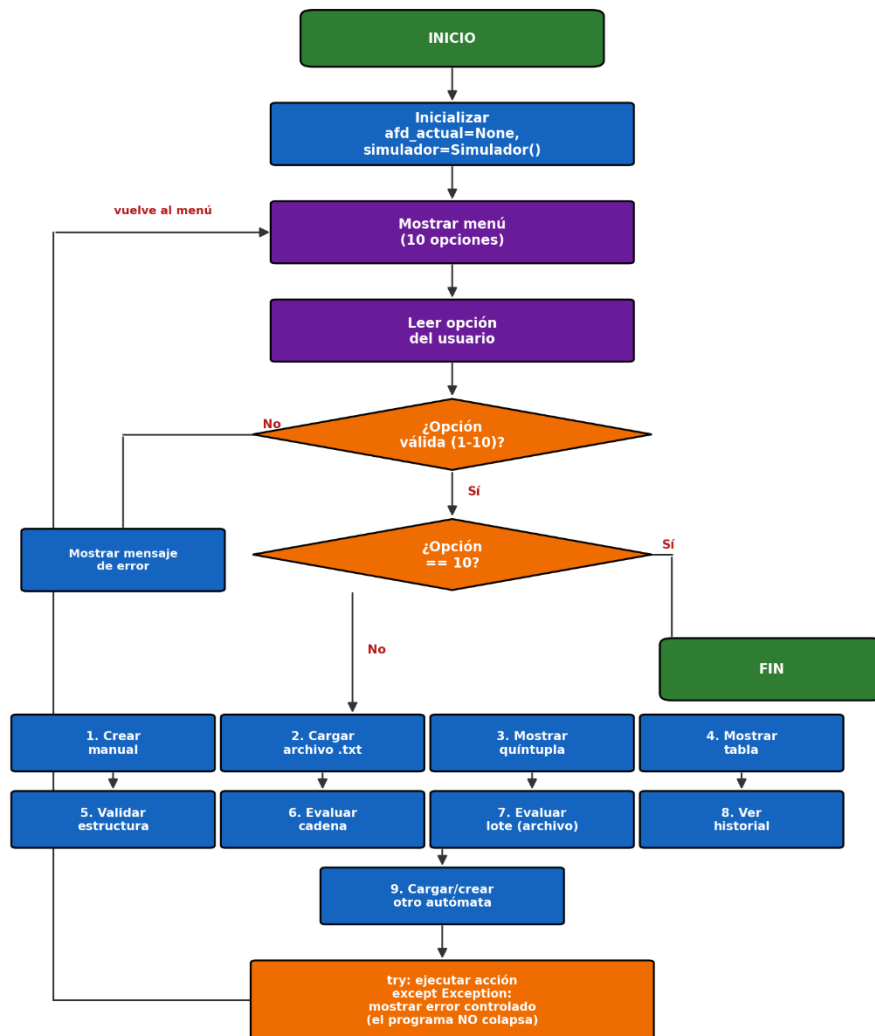
Ruta del archivo .txt a cargar: C:\Users\veget\Downloads\prueba2.txt
[!] Falta la sección obligatoria 'ESTADOS:' en el archivo.

[ERROR] No fue posible construir el AFD a partir del archivo.

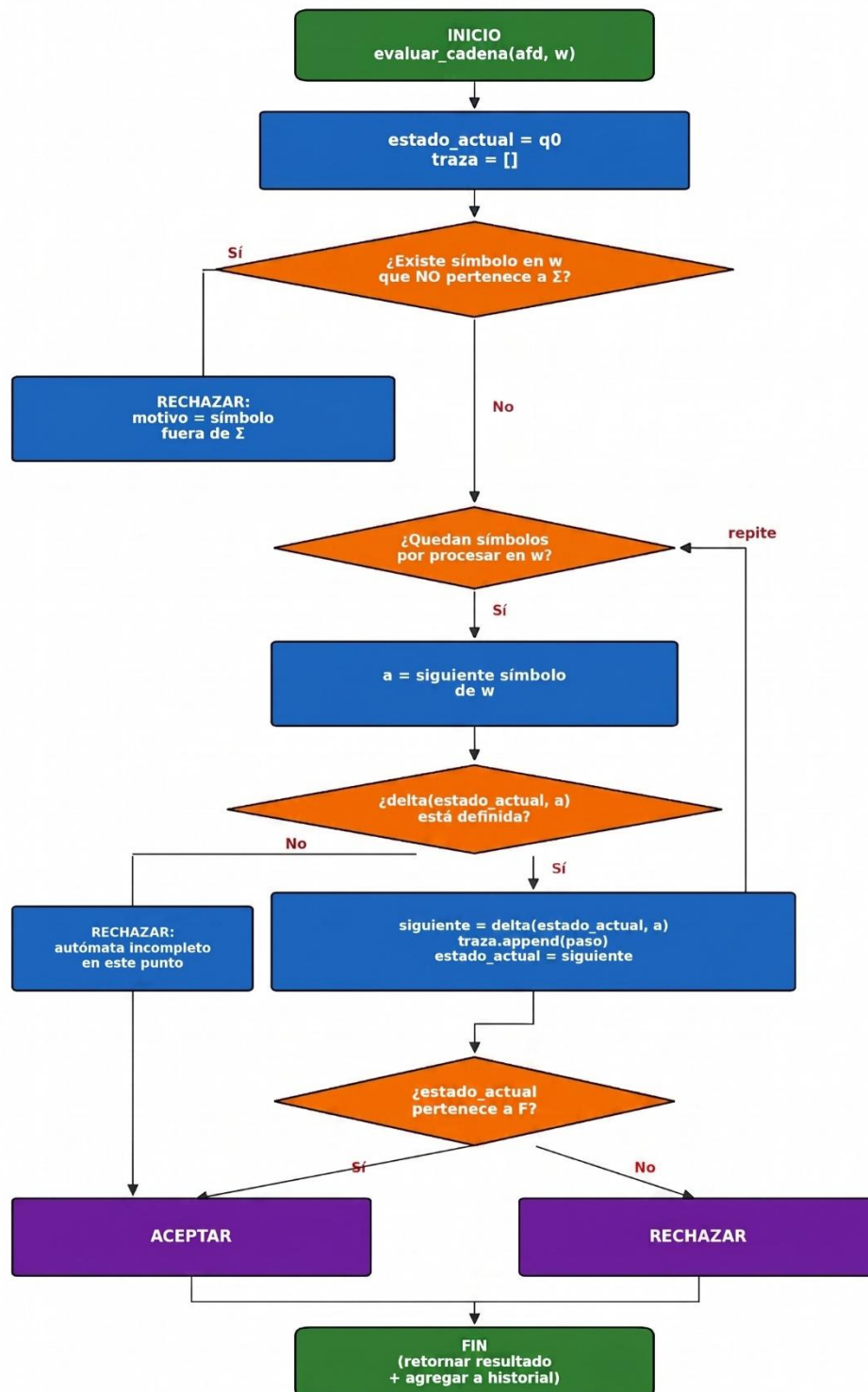
6. Algoritmo (diagramas de flujo)

A continuación, se presenta el diagrama de flujo general del programa (bucle del menú principal) y el diagrama detallado del algoritmo de evaluación de una cadena, que constituye el núcleo de la simulación.

6.1 Flujo general del programa (main.py)



6.2 Algoritmo de evaluación de una cadena (simulador.py)



7. Casos de prueba

Casos utilizados para verificar el manejo de errores exigido por el enunciado, sobre un archivo con errores intencionales:

Caso	Entrada	Resultado esperado
Identificador inválido	ESTADOS: q0,q1,2malo	'2malo' se ignora y se reporta como identificador inválido
Transición múltiple	q0,a,q1 / q0,a,q2	Se reporta transición múltiple para (q0,a); indicio de AFND
Transición-ε	q1,epsilon,q2	Se rechaza con mensaje específico de transición-ε (no AFD)
Línea mal formada	esto no es una linea valida	Se reporta '[Línea N] Transición mal formada' con el número real de línea
Estado inexistente en transición	q9,b,q1 (q9 ∉ Q)	Se reporta como transición inconsistente en la validación de la quintupla
AFD incompleto	Faltan pares (q,a) sin definir	Se listan como 'transición faltante'; se ofrece completar con estado trampa
Cadena con símbolo fuera de Σ	'abx' con Σ={a,b}	RECHAZADA: símbolo 'x' fuera del alfabeto
Cadena aceptada	'ab' sobre el AFD de ejemplo	ACEPTADA: estado final alcanzado ∈ F
Lote de cadenas	ejemplo_cadenas.txt (5 líneas)	Reporte consolidado: 3 aceptadas, 2 rechazadas

Se probó la siguiente cadena:

NOMBRE: AFD_Prueba_Validaciones

ESTADOS: q0,q1,q2,2malo,q0

ALFABETO: a,b,,a

INICIAL: q0

FINALES: q2

TRANSICIONES:

q0,a,q1

q0,a,q2

q0,b,q0

q1,a,q1

q1,epsilon,q2

esto no es una linea valida

q9,b,q1

Y el programa mostró lo siguiente:

```
10. Salir
=====
Seleccione una opción (1-10): 2

Ruta del archivo .txt a cargar: C:\Users\veget\Downloads\afd_prueba_errores.txt
[!] Identificador inválido ignorado: '2malo'.
[!] [Línea 8] Transición múltiple detectada para ('q0', 'a'): ya existía destino 'q1', se ignora el nuevo destino 'q2' (se conserva el primer registrado).
[!] [Línea 11] Transición-ε detectada en 'q1,epsilon,q2': un AFD no admite transiciones que no consuman símbolo (esto corresponde a un AFND/AFN-ε, no a un AFD).
[!] [Línea 12] Transición mal formada (ignorada): 'esto no es una linea valida' (se esperaba 'origen,símbolo,destino').

[OK] AFD 'AFD_Prueba_Validaciones' cargado correctamente desde 'C:\Users\veget\Downloads\afd_prueba_errores.txt'.
```

7.1 Casos adicionales: alfabetos numéricos y de un solo carácter

Casos ejecutados específicamente para verificar que Σ admite símbolos numéricos y para determinar el límite real de longitud de símbolo soportado por el motor de simulación:

Caso	Entrada	Resultado obtenido
Alfabeto binario de 1 carácter	$\Sigma=\{0,1\}$; cadenas '1','01','101','0001','1110','000'	Correcto: 5 aceptadas, 2 rechazadas según pertenencia a F
Alfabeto mixto de 1 carácter	$\Sigma=\{a,b,0,1\}$; cadenas 'ab','1b','a01b', etc.	Correcto: cada cadena se evalúa símbolo por símbolo sin error
Símbolo fuera de Σ (dígito)	$\Sigma=\{0,1\}$; cadenas '012','2'	RECHAZADA: se reporta el dígito '2' como fuera de Σ
Alfabeto MULTI-carácter	$\Sigma=\{10,20,30\}$; cadena '10'	Carga sin error, pero la simulación SIEMPRE rechaza (compara carácter por carácter: '1' y '0' por separado, ninguno pertenece a Σ)
Estados puramente numéricos	ESTADOS: 0,1,2	Los 3 estados se rechazan como identificador inválido; Q queda vacío (comportamiento esperado: los estados deben iniciar con letra o '_')

Conclusión de las pruebas: el alfabeto Σ puede contener dígitos sin problema, siempre que cada símbolo sea de un solo carácter. Los nombres de estado, en cambio, deben seguir la convención de identificador (iniciar con letra o guion bajo), tal como 'q0', 'q1', 'q2'.

Se probó de la siguiente manera:

```
=====
Seleccione una opción (1-10): 4

ESTADO      0      1      a      b
-----
->q0         q0      q1      q1      q0
q1           q0      q1      q1      q2
*q2          q2      q2      q2      q2

Leyenda: '->' estado inicial, '*' estado final, '-' transición no definida.

Ingrese la cadena a evaluar (ENTER para cadena vacía ε): a10baba01011

--- TRAZA DE EJECUCIÓN: 'a10baba01011' ---
[q0] --(a)--> [q1]
[q1] --(1)--> [q1]
[q1] --(0)--> [q0]
[q0] --(b)--> [q0]
[q0] --(a)--> [q1]
[q1] --(b)--> [q2]
[q2] --(a)--> [q2]
[q2] --(0)--> [q2]
[q2] --(1)--> [q2]
[q2] --(0)--> [q2]
[q2] --(1)--> [q2]
[q2] --(1)--> [q2]
Veredicto final: ACEPTADA
Motivo: El estado final de la computación 'q2' pertenece a F.
```

8. Manual de usuario (resumen)

8.1 Ejecución

```
python3 main.py
```

8.2 Flujo recomendado de uso

- Seleccionar la opción 2 e indicar la ruta de un archivo .txt con el formato descrito en la sección 1.1 (o la opción 1 para ingresar el AFD manualmente).
- Usar la opción 3 y 4 para revisar la quintupla y la tabla de transición cargadas.
- Usar la opción 5 para validar la estructura; si el sistema detecta que el AFD está incompleto, aceptar la sugerencia de completar con estado trampa.
- Usar la opción 6 para evaluar cadenas individuales con su traza, o la opción 7 para evaluar un archivo completo de cadenas.
- Usar la opción 8 en cualquier momento para revisar el historial acumulado de la sesión.
- Usar la opción 9 para cambiar de autómatas sin reiniciar el programa, o la opción 10 para salir.

8.3 Requisitos

- Python 3.8 o superior.
- No requiere librerías externas: solo se usan os, re, sys y typing de la biblioteca estándar.

8.4 Advertencia sobre el alfabeto Σ

- Cada símbolo de Σ debe ser de un único carácter (letra, dígito o signo). Alfabetos numéricos como $\Sigma=\{0,1\}$ son válidos y funcionan correctamente. Símbolos de más de un carácter (p. ej. '10') se cargan sin error pero nunca serán reconocidos durante la simulación.